

Running a cluster (active-passive HA)

Early Access · MessageFoundry v0.3.2 · September 2026

Status: built (Track B). Single-node operation remains the default, with unchanged behavior. Set `[cluster].enabled = true` to enable clustering on PostgreSQL **or** SQL Server. Clustering is the **active-passive** (leader/standby failover) HA model — the supported HA mode. The horizontal **active-active** scale-out path (the graph running concurrently on every node) was **dropped (2026-06-18) and its code removed**. It is not a planned milestone. Design records: the cluster ADRs **0005 / 0006** (the converged data) and **ADR 0008** (the observability API below). Code: `pipeline/cluster.py` (PostgreSQL) + `pipeline/cluster_sqlserver.py` (SQL Server).

MessageFoundry provides high availability (HA) through **N identical engine processes sharing one server database**, either PostgreSQL or SQL Server. One leader runs the graph. The other nodes remain warm standbys.

The database holds the durable staged queue, row leases, leader election, and shared configuration/state versions. No separate broker is required. Single-node operation remains the default. Enable `[cluster]` to coordinate nodes and let a standby take over after failure.

Requirements

A clustered deployment **requires** (enforced at config load):

- `[cluster].enabled = true`
- `[store].backend = "postgres" or "sqlserver"` — SQLite is single-file/single-node. The cluster needs the shared `nodes` table + row leases a server DB provides. Both backends run the same active-passive leadership lease (`pipeline/cluster.py` for Postgres, `pipeline/cluster_sqlserver.py` for SQL Server).
- `[store].pool_size >= 2` — a clustered node runs concurrent membership, lease-renewal, reclaim, and stage workers. Allow extra connections for that work (prefer `>= 3`).

Every node points at the **same** server database (same `[store]` server/database/schema) and runs the **same** config dir.

```
# messagefoundry.toml – identical on every node (the DB password comes from MEFOR_STORE_PASSWORD)
[store]
backend   = "postgres"   # or "sqlserver"
server    = "db.internal"
database  = "messagefoundry"
username  = "mefor"
pool_size = 40           # default (ADR 0062); >= 2 under [cluster]
```

```
[cluster]
enabled = true
# node_id is auto-derived (host:pid:hex, reusing the store's lease owner-id) – pin it only for a
# stable identity across restarts or in tests.
heartbeat_seconds = 10.0
node_timeout_seconds = 30.0 # a node is "dead" when last_seen is older than this; must be > heartbeat
# How often the leader runs the RECURRING background expired-lease reclaim sweep (the active-passive
# background lease-reclaim). It does NOT gate failover speed: on promotion the new leader recovers the
# prior leader's stranded rows immediately (owner-scoped, lease-blind; #293), so [store].lease_ttl_seconds
# (the per-row lease TTL, default 60s) is the background-sweep ceiling, NOT the failover-recovery driver.
reclaim_interval_seconds = 30.0
# Leadership lease (active-passive self-fencing). Timing invariant enforced at load:
# heartbeat_seconds < leader_fence_timeout_seconds < leader_lease_ttl_seconds
leader_lease_ttl_seconds = 30.0 # a standby acquires leadership only once the lease has expired
leader_fence_timeout_seconds = 20.0 # a leader that can't renew within this self-fences (split-brain
guard)
# --- Leader preference (ADR 0096) – per-node; default (0.0, true) = unweighted first-lease-wins -----
# acquire_delay_seconds: seconds this node waits PAST the lease-expiry time before it may take over an
# EXPIRED lease (handicap). A preferred site keeps 0.0; a warm remote-DR node sets a positive value so
# a preferred node wins the routine take-over race. NEVER delays a renewal by the current leader, and
# only ever makes a node claim LATER – so it can't open a two-leader window. Governs take-over of an
# EXPIRED lease only; the very first election on an empty table is a plain race.
acquire_delay_seconds = 0.0
# promotable: false = this node may NEVER become leader (never inserts/takes-over/renews the lease); a
# node that somehow already leads steps down cleanly on its next tick. Use it for a warm, passive DR
# engine. At least ONE promotable node MUST exist, or no node ever acquires the lease and the graph
# never drains (an all-non-promotable cluster is a misconfiguration).
promotable = true
```

Warm DR: run a remote DR-site engine as a **non-promotable cluster member** (`promotable = false`) — NOT as a `[dr].activate` box. Combining `[dr].activate` with `[cluster]` is refused at config load (the DR run-profile gates which connections start, not lease acquisition, so a lease-contending DR box could win leadership and drive the primary store cross-WAN).

Start the same `serve` command on each host/process — e.g.:

```
python -m messagefoundry serve --service-config messagefoundry.toml --config ./config
```

(For a local DEV Postgres, `scripts/dev/postgres.ps1` sets the `MEFOR_STORE_*` connection env and `MEFOR_ALLOW_INSECURE_TLS=1` for a loopback, no-TLS database — DEV convenience only.)

What each node does

The **leader (primary)** runs the whole message graph. Every other node remains a **warm standby** and competes for leadership. The cluster coordinates these operations:

- **Active-passive graph gating (Workstream A1).** Only the leader runs listeners (MLLP, TCP, File, and others) and Router, transform, and delivery workers. A standby runs membership heartbeats and cache convergence, but no listeners or workers. It starts the graph after it acquires leadership and stops the graph after it loses

leadership.

The graph supervisor checks leadership at short intervals. A demotion or self-fence stops new intake and processing. The self-fencing leadership lease prevents the old leader from processing before a standby can acquire leadership. Row leases provide another safeguard for the recurring reclaim sweep, which recovers only expired leases.

Clients reconnect through a floating VIP or load-balancer health check. On promotion, the new leader immediately recovers rows owned by another instance, never its own rows. This owner-scoped recovery ignores row-lease expiry because the old leader has already stopped under the self-fencing guarantee.

Delivery therefore does not wait for `[store].lease_ttl_seconds`, whose default is 60s. That wait previously caused the approximately 60s PostgreSQL recovery delay (#293). The recurring background sweep still checks row-lease expiry.

- **Leader election (self-fencing lease).** Exactly one node holds the `leader_lease` row. It renews the lease every `heartbeat_seconds` to `DB_now + leader_lease_ttl_seconds`. The database clock determines expiry, so node clock differences do not affect leadership.

A standby can acquire only an expired lease. If the leader cannot renew within `leader_fence_timeout_seconds`, it stops acting as leader. That timeout is less than the lease TTL, so the old leader stops before a standby can take over. A clean shutdown expires the lease for prompt takeover.

- **Store-checked leader epoch (fencing token).** The temporal self-fence requires the old leader to detect a pause or partition before its lease expires. The `leader_lease` row also carries a monotonic `leader_epoch` as a database safeguard. Only a fresh leadership acquisition increases this value. Renewal does not change it.

On promotion, the engine passes the coordinator's epoch to `Store.set_leader_epoch`. Each FIFO claim transaction checks `held >= leader_lease.leader_epoch`. If an old leader resumes after the temporal timeout, its stale epoch permits **0 rows**. Its `UPDATE` matches nothing, so it delivers nothing.

The current leader has a matching epoch and claims normally. This check rejects stale claims without changing valid FIFO order. It applies to PostgreSQL and SQL Server only. SQLite uses one active node, so `set_leader_epoch` has no effect and its claim behavior is unchanged.

The column migration is additive: `ADD COLUMN IF NOT EXISTS` or a guarded `ALTER` under the DDL lock. It supports an in-place cluster upgrade. Existing rows receive `0`, and the first fresh acquisition increases the value to `1`.

- **Leader-gated WRITE singletons.** Retention purges and the lease-reclaim sweep run **only on the leader**, so they never double-execute.
- **Leader-gated poll-source intake.** Only the leader polls shared resources, such as watched directories, database tables, or remote directories. The standby does not run the graph. The poll loop also checks leadership as a separate safeguard.

- **Per-lane FIFO survives failover.** Only the leader processes the graph. The ordinary FIFO claim (`claim_next_fifo`) recovers a stranded lane-head row with an expired lease before the head SELECT. Both operations use the same transaction. The stranded head blocks later rows, so they cannot pass it. This replaced the removed active-active per-lane lease mechanism.
- **Reference, configuration, and transform-state convergence.** The leader builds each reference-set snapshot, and followers read it from the shared database. A configuration reload on one node increases a shared version token. Other nodes then reload their own configuration directories. Transform-state writes use a separate version token for each namespace.

Observability — `/cluster/status` and `/cluster/nodes`

Two read-only API endpoints expose membership and leadership. Both require `Permission.MONITORING_READ`, held by VIEWER and higher roles. Neither exposes PHI or adds a permission. Use the console or any API client. `/cluster/status` reads memory. `/cluster/nodes` reads the `nodes` table once.

GET `/cluster/status` — this node's posture

```
{
  "node_id": "node-a:4812:1f9c2a7b",
  "clustered": true,
  "is_leader": false,
  "role": "standby",
  "config_version": 7
}
```

`role` identifies the node for operators and load-balancer checks:

- `"primary"`: the leader runs the graph.
- `"standby"`: the follower has no listeners or workers.
- `"single-node"`: clustering is disabled.

A single node reports `clustered: false`, `is_leader: true`, `role: "single-node"`, and `config_version: 0`:

```
{ "node_id": "host:1234:ab12cd34", "clustered": false, "is_leader": true,
  "role": "single-node", "config_version": 0 }
```

GET `/cluster/nodes` — all nodes + the derived leader

`leader_node_id` identifies the single **live** leader. The freshness check excludes a crashed former leader when `last_seen` exceeds `node_timeout_seconds`. A stale leader flag does not override that check.

Two-node cluster:

```
{
  "nodes": [
    { "node_id": "node-a:4812:1f9c2a7b", "host": "node-a", "pid": 4812,
      "status": "active", "started_at": 1750000000.0, "last_seen": 1750000123.4, "is_leader": true,
```

```

    "acquire_delay_seconds": 0.0, "promotable": true },
  { "node_id": "node-b:5210:7c3e9d10", "host": "node-b", "pid": 5210,
    "status": "active", "started_at": 1750000005.0, "last_seen": 1750000124.1, "is_leader": false,
    "acquire_delay_seconds": 15.0, "promotable": false }
],
"leader_node_id": "node-a:4812:1f9c2a7b",
"lease_owner": "node-a:4812:1f9c2a7b",
"lease_expires_at": 1750000153.4
}

```

`lease_owner` and `lease_expires_at` come from the authoritative `leader_lease` row. They identify the lease holder and expiry time on the database clock. After expiry, a standby can acquire the lease if the leader does not renew it.

`lease_owner` normally equals the heartbeat-derived `leader_node_id`. A brief difference during failover is expected. The lease determines which node may process messages.

Each node also reports its leader-preference settings (ADR 0096). `acquire_delay_seconds` delays acquisition of an expired lease, with `0.0` meaning no delay. `promotable: false` prevents a standby from becoming leader.

A single node has no heartbeat history, so `started_at` and `last_seen` are `null`. It remains leader, with `lease_expires_at: null`:

```

{
  "nodes": [
    { "node_id": "host:1234:ab12cd34", "host": "host", "pid": 1234,
      "status": "active", "started_at": null, "last_seen": null, "is_leader": true,
      "acquire_delay_seconds": 0.0, "promotable": true }
  ],
  "leader_node_id": "host:1234:ab12cd34",
  "lease_owner": "host:1234:ab12cd34",
  "lease_expires_at": null
}

```

A clean shutdown leaves a `status: "left"` record and clears the leader flag. A crashed node stops updating `last_seen`, so the freshness check excludes it. During failover, the freshest live heartbeat wins if both old and new leader flags remain set. The result names at most one leader and never names a dead node.

`/cluster/status` reads the node's own leadership gate and is authoritative for that node. `/cluster/nodes` uses heartbeat flags and can lag by one `heartbeat_seconds` interval.

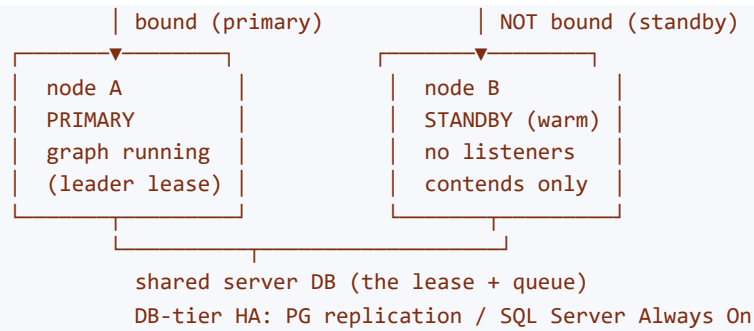
After failover, the new primary can report `is_leader: true` before `/cluster/nodes` shows its `leader_node_id`. A temporary `leader_node_id: null` during that interval reflects the heartbeat delay.

Deployment topology (active-passive)

```

MLLP/TCP senders → [ floating VIP / load balancer
                      health check = TCP connect to the listener port
                      (only the PRIMARY binds it, so the VIP always lands on it) ]
(partners)

```



One primary processes. The rest are warm standbys. All nodes point at the **same** server DB and run the **same** config dir. The `leader_lease` row elects exactly one primary, which alone binds listeners and runs workers (A standby binds nothing). DB-tier high availability (a replica / failover) is **delegated to the database** (PostgreSQL streaming replication, SQL Server Always On) — MessageFoundry does not replicate the store itself.

Client reconnect — a floating VIP / LB health check is REQUIRED

Senders connect through a **floating VIP or load balancer**. After failover, they reconnect through that address to reach the new primary:

Planned Windows-only alternative. ADR 0056 proposes optional engine ownership of the VIP. The engine would move it with the leadership lease, without an external LB, VRRP, or WSFC. This feature is **not built**. Until it is available, use the external VIP or load balancer described here. Linux and container deployments will still require that external option. It remains the cross-platform default and recommended option for the strictest split-brain guarantee.

- **MLLP / TCP inbound (per listener).** Use a VIP per inbound port whose health check is a **TCP connect to that port**. Because only the **primary** binds the port (the active-passive graph gating), the check passes only on the primary, so the VIP routes inbound traffic to it automatically. On failover the new primary binds the port, the old one's closes, and the VIP follows. MLLP senders see a connection drop and reconnect through the VIP — make partners **reconnect on drop** (standard MLLP client behavior).
- **Engine API edge (console / IDE).** The control/read API runs on **every** node against the shared database. An API VIP can check liveness through unauthenticated `GET /health`. To pin operations to the primary, read `GET /cluster/status → role ("primary" / "standby")`, or `GET /cluster/nodes → leader_node_id + lease_owner` (the console surfaces the live primary).

Failover is not instantaneous

Failover includes a promotion window. Measure it with the Workstream-D failover benchmark before planning availability requirements:

- **Clean stop** (graceful shutdown / planned switchover): the leaving primary **expires its lease**, so a standby acquires on its next heartbeat — failover is prompt ($\approx$ one `heartbeat_seconds`).

- **Crash / partition:** the primary's lease **ages out**, so a standby acquires after up to `leader_lease_ttl_seconds`. A partitioned old primary **self-fences** within `leader_fence_timeout_seconds` (< the TTL), so it stops processing before the standby takes over.
- During the window, in-flight rows are protected by the **row leases** (a standby reclaims only *expired* leases). The new primary runs an owner-scoped recovery **once on promotion** to recover the dead primary's in-flight rows promptly (and the ordinary FIFO claim reclaims a stranded lane head, so order survives). At-least-once delivery + idempotent re-runs mean a row interrupted mid-delivery is re-delivered after its lease expires (so downstream connections must stay idempotent).

Tune the lease timings to your network

The defaults (`heartbeat_seconds=10` , `leader_fence_timeout_seconds=20` , `leader_lease_ttl_seconds=30`) trade a ~30 s crash-failover for ample margin. Lower all three proportionally (keeping `heartbeat < fence < ttl`) for faster failover at the cost of less tolerance for a slow DB / GC pause. Because the **leadership** lease is evaluated on the **database's** clock, node clock skew does not affect who may hold leadership. The **row** leases, however, use node wall-clock — see below.

Operational assumptions (honor these)

1. **Clock sync (NTP).** Row leases are wall-clock — keep node clocks reasonably synced so a lease expiry is not mistimed across nodes.
2. **Identical config on every node.** Each node loads the graph (Connections / Routers / Handlers) from its **own** config dir. Convergence coordinates the reload *version*, not the files. Deploy the same config dir to all nodes.
3. **Coordinated config changes.** Apply a config change as a **coordinated (not rolling) restart**, so nodes do not run divergent graphs across the change window.

Related

- [ADR 0008](#) — the observability API design.
- [docs/adr/](#) — the cluster ADRs and the staged-pipeline / store architecture they build on.
- [docs/CONFIGURATION.md](#) — the full `[store]` / `[cluster]` settings catalog.